

UNIVERSITY OF RWANDA

College of Education

Department of Mathematics and Computer Science

Module&Code: CS80541 - Mobile Applications Programming

D&G E-COMMERCE MOBILE APPLICATION TESTING REPORT

Student Name: Mr Samuel NTAKIRUTIMANA

Student ID: 224006364

Lecturer: Mr UWAMAHORO Bonaventure

Submission Date:……

Declaration

I, Mr. Samuel NTAKIRUTIMANA 224006364, hereby declare that this report is the result of my own work. All sources of information, code libraries and ideas that are not my own have been acknowledged in the text and listed in the References section. This work has not been submitted before, in whole or in part, for any other assessment.

Signature: Mr Samuel NTAKIRUTIMANA Date: 2026-Oct-25th

Acknowledgement

I would like to express my sincere gratitude to Mr UWAMAHORO Bonaventure for his guidance throughout this course. His teaching on modern mobile development, on breaking a problem into manageable parts and on designing Mobile Apps

I also thank the university for providing a good learning environment, my classmates for the many discussions and for helping me test the application on different devices, and my family for their constant support and encouragement.

Finally, I acknowledge the open-source communities behind React Native, Expo, Axios, FastAPI and SQLAlchemy, and the Cloudinary platform, whose tools and documentation made this work possible.

Abstract

D&G is an online retail start-up in Rwamagana, Rwanda, that sells clothes, shoes, bags, household equipment, jewellery and cosmetics. Before this project the business depended on physical visits and informal telephone or WhatsApp messages, which limited its customer reach, made order handling manual, and gave customers no reliable view of stock or delivery progress.

This report describes the design and implementation of the D&G E-Commerce Mobile Application, a cross-platform application for Android and iOS built with React Native 0.88, Expo SDK 58 and TypeScript. State is managed with React Context (authentication, cart and language) combined with in-memory screen caches that make tab navigation instant. The application communicates with a FastAPI backend deployed on Render, backed by PostgreSQL, through an Axios client that attaches a JWT bearer token to every request. Product images are stored on the Cloudinary CDN.

The main functions developed are: secure registration, login, password recovery and account deletion; product browsing with category filters, debounced search, Rwandan Franc (RWF) price tiers and swipeable multi-image galleries; a persistent shopping cart; a two-step checkout with Rwandan delivery districts and simulated MTN Mobile Money, Airtel Money and bank payments; order history with a five-second polling Two-Way Delivery Handshake between customer and administrator; automatic inventory control; three-language support (English, Kinyarwanda and French); and an administrator suite for catalogue management and sales reporting.

The application was verified with zero TypeScript compilation errors and tested on an Android emulator, a physical Android device and an iOS simulator. The separate Testing Report shows that all 44 test cases passed on the final build. The result is a working, secure and localised shopping channel that D&G can use to reach customers beyond Rwamagana.

Keywords: React Native, Expo Router, e-commerce, FastAPI, PostgreSQL, Rwandan Franc (RWF), Two-Way Delivery Handshake, REST API, Cloudinary

Table of Contents

Declaration i
Acknowledgement ii
Abstract iii
Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 General Objective 2
1.4 Specific Objectives 2
1.5 Scope of the Project 2
1.6 Technologies Used 3
1.7 Organisation of the Report 4
Chapter 2: Requirements Analysis 4
2.1 Functional Requirements 4
2.2 Non-Functional Requirements 5
2.3 User Role Analysis 6
2.4 Requirements Traceability 6
Chapter 3: System Design 6
3.1 System Architecture 7
3.2 Use Case Diagram 8
3.3 Activity Diagrams 8
3.3.1 Login Process 8
3.3.2 Checkout Process 9
3.3.3 Order Tracking Process 10
3.4 Navigation Design 12
3.5 Data Models 13
User 13
Product 13
Order 13
3.6 Backend API Overview 14
Chapter 4: Implementation 15
4.1 Project Setup 15
4.2 Authentication Module 15

4.2.1 Login 16
4.2.2 Registration 16
4.2.3 Forgot Password 17
4.2.4 Logout 17
4.3 Product Browsing Module 17
4.3.1 Product List 17
4.3.2 Search 18
4.3.3 Filter 19
4.3.4 Product Details 19
4.4 Shopping Cart Module 20
4.5 Checkout Module 21
4.6 Order Management Module 22
4.7 API Integration 24
4.8 Multi-language Support 24
4.9 Navigation and Access Control 25
4.10 State Management 25
4.11 Validation, Error Handling and User Feedback 26
Chapter 5: Reports Generated by the System 26
5.1 Product Listing Report 26
5.2 Customer Information Report 26
5.3 Cart Summary Report 26
5.4 Order History Report 27
5.5 Sales Summary Report 28
5.6 Application Performance Report 28
Chapter 6: Challenges and Solutions 29
Chapter 7: Conclusion and Recommendations 29
7.1 Conclusion 29
7.2 Future Improvements 30
References 30
Appendices 31
Appendix A: Important Code Snippets 31
Appendix B: Installation Guide 31

List of Figures

Overall system architecture....7
Use case diagram of the D&G application....8
Activity diagram: login process....9
Activity diagram: checkout process....10
Activity diagram: order tracking process....11
Navigation structure of the application....12
Login screen....16
Registration screen....17
Product list screen....18
Search by product name....18
Filtering products by category and price....19
Product details screen....20
Shopping cart screen....21
Checkout screen: delivery and payment....22
Order history screen....23
Order details and tracking status....23
Language switching: translated version (Ikinyarwanda)....25
Cart summary report....27
Order history report....28

List of Tables

Project scope 2
Technologies and libraries 3
Functional requirements 4
Non-functional requirements 5
Assignment tasks mapped to report sections 6
Screens and access rules 12
API endpoints used 14
State management contexts 25
Challenges and solutions 29
Objectives versus achievements 30
Key source files 31

Chapter 1: Introduction

This chapter introduces the D&G e-commerce mobile application. It explains the business background, the problem the application solves, the objectives, the scope of the work and the technologies used, and it closes with a guide to the remaining chapters.

1.1 Background

E-commerce is growing quickly across Sub-Saharan Africa. Wider 4G and 5G coverage, cheaper smartphones and the spread of mobile money services such as MTN Mobile Money and Airtel Money have made it possible for ordinary shoppers to browse, pay and receive goods without visiting a shop. Mobile commerce matters most in this region because the smartphone, not the desktop computer, is the main way people reach the internet.

Rwanda is well placed to benefit from this shift. The national agenda for a cashless economy and for smart retail has encouraged businesses to move their sales channels online and to accept electronic payments.

D&G is a retail start-up located in Rwamagana in the Eastern Province of Rwanda. It sells a wide range of fashion and household goods:

- Clothes (men and women apparel)
- Shoes (sneakers and formal footwear)
- Bags (handbags and backpacks)
- Household equipment (kitchenware and small appliances)
- Jewellery (watches, necklaces and bracelets)
- Cosmetics (skincare and beauty products)

D&G wants to strengthen its presence in the digital marketplace by offering customers a simple and reliable shopping experience on their phones.

1.2 Problem Statement

D&G currently serves customers through physical visits and informal messages on social media and WhatsApp. This approach creates several problems:

- The shop cannot show customers real-time stock counts, so customers are disappointed when an item has already run out.
- Orders are coordinated by hand, with no structured way to record delivery details or follow a delivery.
- Customers who do not read English, or who think in another currency, find it difficult to understand prices and product information.
- Sold-out products often remain visible, so the catalogue does not match what is really available.

A cross-platform mobile application connected to a central REST API solves these problems. One code base serves both Android and iOS users, prices are shown in Rwandan Francs, the interface is available in three languages, and stock and orders are managed in one place.

1.3 General Objective

To design, develop, test and document a robust, secure and user-friendly cross-platform e-commerce mobile application for D&G using React Native, TypeScript and a FastAPI REST backend, which allows customers to browse localised products, complete transactions in Rwandan Francs (RWF) and track their orders through a live delivery handshake.

1.4 Specific Objectives

1. To implement isolated user authentication: registration, login, password recovery, secure JWT token storage and cascading account deletion.
2. To develop a product catalogue with category filtering, debounced search, RWF price tiers, multi-image viewing and instant switching between English, Kinyarwanda and French.
3. To build a persistent shopping cart with quantity controls and subtotal calculation, and a two-step checkout with delivery routing and Rwandan payment simulation.
4. To implement real-time order tracking through a five-second polling Two-Way Delivery Handshake.
5. To create an automated inventory control engine that decrements stock on purchase and removes depleted products.
6. To build an administrative suite for product management, delivery coordination and sales analytics.
7. To test the application on Android and iOS emulators and devices for functionality, performance, security and error handling.

1.5 Scope of the Project

Table 1 states what the project includes and what it deliberately leaves out, so that the boundaries of the assignment are clear.

Table 1: Project scope

Included in the project	Not included in the project
Customer registration, login, logout and password recovery	Third-party supplier ERP integration
Role-based dual interface (customer and store administrator)	Physical barcode or RFID warehouse scanner integration
Product browsing, debounced search, category and RWF price filtering	Live production merchant bank contract (payment is simulated through MTN, Airtel and bank hooks)

Included in the project	Not included in the project
Swipeable multi-image product details with stock warnings	Customer-to-customer marketplace functionality
Cloudinary multi-image upload from the device photo library	Automated drone delivery routing
Shopping cart with backend database persistence	Customer credit scoring or buy-now-pay-later
Two-step checkout with Rwandan districts and payment options	
Real-time order tracking and five-second Two-Way Delivery Handshake	
Automatic inventory decrement and removal at zero stock	
Three-language engine (English, Kinyarwanda, French)	
Administrative sales KPIs, revenue reports and catalogue CRUD	
Secure cascading account deletion	

1.6 Technologies Used

Table 2 lists the technologies and libraries used on the client and server side, with the exact versions in the project.

Table 2: Technologies and libraries

Technology / library	Purpose	Version
React Native	Native cross-platform mobile runtime	0.88.0
React	Component architecture and hooks	19.3.0
Expo	Managed toolchain and native modules	~58.0.17
Expo Router	File-based, typed screen navigation	~58.0.25
TypeScript	Static typing and compile-time safety (checked with tsc --noEmit)	~6.0.3
Axios	HTTP client for REST API communication	^1.20.0
AsyncStorage	Persistent on-device key-value store for the JWT token	2.2.0
Expo Image Picker	Selecting images from the gallery for upload	~58.0.10
Expo Safe Area Context	Handling notches, home indicators and Android soft keys	~5.10.1
Ionicons (@expo/vector-icons)	Icons for tabs, badges and buttons	^15.0.2
UUID	Unique client-side identifiers	^14.0.3
FastAPI (backend)	Asynchronous REST API (Python 3.11)	0.110.0+
SQLAlchemy	Object relational mapper for database models	2.0+

Technology / library	Purpose	Version
PostgreSQL / SQLite	Production database / local fallback (dg.db)	PostgreSQL 16 / SQLite 3
Cloudinary	Cloud CDN for multi-image storage	API v1_1

1.7 Organisation of the Report

Chapter 2 analyses the functional and non-functional requirements and the user roles, and maps each task in the assignment brief to the report. Chapter 3 presents the system design: architecture, use cases, activity diagrams, navigation, data models and the backend API. Chapter 4 describes the implementation of each module with code and screenshots. Chapter 5 presents the reports generated by the system. Chapter 6 discusses the challenges met and how they were solved, and Chapter 7 concludes with a review of the objectives and suggestions for future work. The appendices contain code references and the installation guide. Testing is documented in a separate Testing Report.

Chapter 2: Requirements Analysis

This chapter states what the system must do and the qualities it must have. The requirements were derived from the assignment brief, the provided backend API and an analysis of how a typical online shopper in Rwanda buys and receives goods.

2.1 Functional Requirements

Table 3 lists the functional requirements. Each has an identifier that is reused in the Testing Report.

Table 3: Functional requirements

ID	Requirement	Priority	Module
FR-01	The system shall allow new users to register with an email address and password, validating the email format and minimum password length.	High	Authentication
FR-02	The system shall authenticate existing users, return a JWT bearer access token and persist it in device storage.	High	Authentication
FR-03	The system shall provide a password reset request flow that does not reveal whether an email is registered.	Medium	Authentication
FR-04	The system shall allow users to browse products, filter by category, search by keyword and filter by RWF price tier.	High	Products
FR-05	The system shall display product details, including a swipeable multi-image carousel, RWF price, stock badge and category.	High	Products
FR-06	The system shall let users switch between English, Kinyarwanda and French on all screens without reloading.	Medium	Multilingual

ID	Requirement	Priority	Module
FR-07	The system shall let users add items to the cart, adjust quantities, calculate subtotals in RWF and persist the cart to the backend.	High	Cart
FR-08	The system shall provide a two-step checkout collecting delivery details (full name, phone, WhatsApp, district) and payment through MTN MoMo, Airtel Money or bank transfer.	High	Checkout
FR-09	The system shall record orders, show order history and display delivery status (In Transit or Delivered).	High	Orders
FR-10	The system shall provide a Two-Way Delivery Handshake allowing either the customer or the administrator to confirm delivery, synchronised in real time.	High	Deliveries
FR-11	The system shall decrement stock on each successful order and delete a product when its quantity reaches zero.	High	Inventory
FR-12	The system shall give administrators product CRUD with multi-image upload to Cloudinary.	High	Admin
FR-13	The system shall let users permanently delete their account after email and password re-verification, cascading to orders and cart records.	High	Security

2.2 Non-Functional Requirements

Table 4 describes the quality expectations and how the application meets each one.

Table 4: Non-functional requirements

Category	Requirement	How it is achieved
Security	Secure communication, tokenised authorisation, credential hashing	Bcrypt password hashing (passlib); OAuth2 bearer JWT (HS256) injected by an Axios interceptor; secrets kept in .env files.
Performance	Instant screen loading and low perceived latency	In-memory caching on all five tab screens gives 0 ms tab navigation without spinners; search is debounced by 300 ms.
Reliability	Fault tolerance and offline resilience	Unified Axios error handler (getApiErrorMessage), try/catch with friendly alerts, retry prompts and database foreign-key cascading.
Data integrity	Accurate inventory	Stock is decremented in the backend (crud_order.py) and pre-validated in the client (checkout.tsx); products at zero stock are removed.
Usability	Rwandan market localisation	Prices in RWF, Rwandan district picker, trilingual interface (EN, RW, FR) and separate customer and admin interfaces.
Compatibility	Consistent behaviour on Android and iOS	Responsive layouts using react-native-safe-area-context for Android soft keys and the iOS home indicator.

2.3 User Role Analysis

The application has two roles, identified by the is_admin flag returned by the API.

Customer. The customer browses the catalogue, views multi-image galleries and filters by category and RWF price. The customer adds items to the cart, checks out with Rwandan delivery routing and payment options, sees a delivery-in-transit banner and confirms receipt of the package. The customer can also change the language, view order history and delete the account.

Administrator. The administrator has a dedicated dashboard showing total revenue in RWF, pending orders and catalogue size. The administrator creates, updates and deletes products with Cloudinary multi-image upload, manages deliveries using the customer contact details (phone, WhatsApp, district), and views sales, inventory distribution and fulfilment-rate reports.

2.4 Requirements Traceability

Table 5 maps each task in the assignment brief to the source files that implement it and to the report section that explains it.

Table 5: Assignment tasks mapped to report sections

Task	Assignment requirement	Implemented in source code	Section
1	Product browsing, search, filter, details, languages	CustomerHome.tsx, CustomerSearch.tsx, [id].tsx, i18n.ts	4.3, 4.8
2	Registration, login, logout, password recovery	(auth)/login.tsx, register.tsx, forgot-password.tsx, AuthContext.tsx	4.2
3	Shopping cart and checkout	CustomerCart.tsx, CartItemRow.tsx, checkout.tsx, CartContext.tsx	4.4, 4.5
4	Order history, details and tracking	CustomerOrders.tsx, AdminConfirmOrders.tsx, crud_order.py	4.6
5	API integration, error handling, loading states	services/api.ts, Axios interceptors, getApiErrorMessage	4.7, 4.11
6	Application testing on Android and iOS	Emulator and device test matrix, TypeScript verification	Testing Report
7	Reports and documentation	AdminReports.tsx, AdminDashboard.tsx, API_REFERENCE.md	Chapter 5
8	Navigation and state management	(tabs)/_layout.tsx, AuthContext, CartContext, LanguageContext	4.9, 4.10

Chapter 3: System Design

This chapter shows how the system was planned before and during coding: the overall architecture, the use cases, the main activity flows, the navigation structure, the data models and the backend API.

3.1 System Architecture

The system has four tiers. The client layer is the React Native application, which contains the user interface screens, the global state contexts, the in-memory screen caches and the Axios HTTP client. The server layer is a FastAPI application on Render, organised into routers for authentication, users, products, carts and orders, protected by a JWT bearer guard and connected to the database through the SQLAlchemy ORM. The database layer is PostgreSQL. Two external services complete the picture: Cloudinary stores product images, and the simulated payment hooks represent the MTN Mobile Money, Airtel Money and bank channels.

Data flows as follows. A user action on a screen updates the relevant context, and the Axios client sends a REST request with the bearer token. The server validates the token, performs the operation through the ORM and returns JSON, which the client stores in context and renders. Images are uploaded directly from the device to Cloudinary, and only the returned image URL is sent to the API.

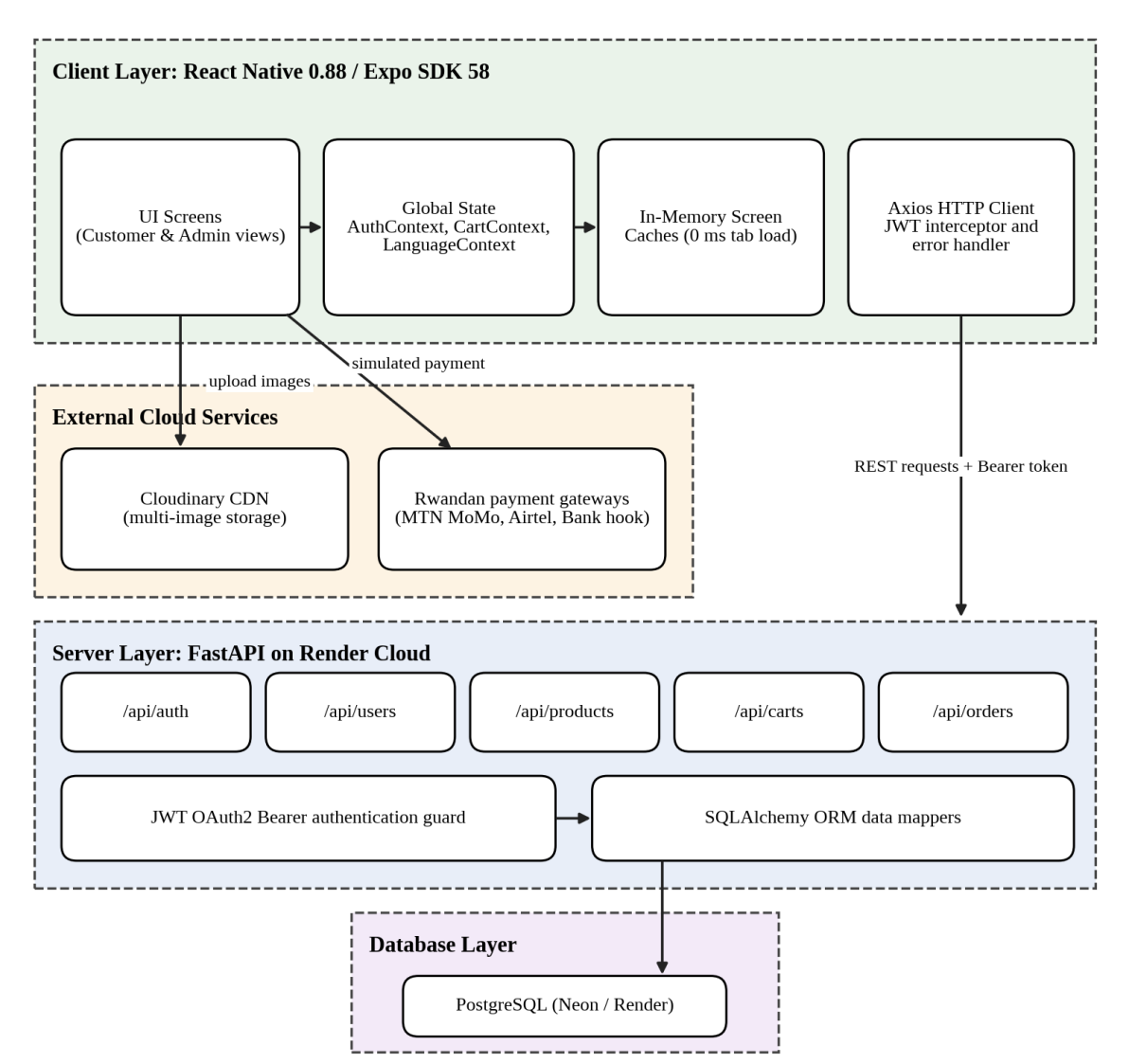


Figure 1: Overall system architecture

3.2 Use Case Diagram

Figure 2 shows the two actors and their use cases. Both actors register and log in (UC1). The customer additionally browses, filters, views products, switches language, manages the cart, checks out, tracks orders and can delete the account (UC2 to UC9). The administrator views analytics, manages products, confirms deliveries and reads sales and inventory reports (UC10 to UC13).

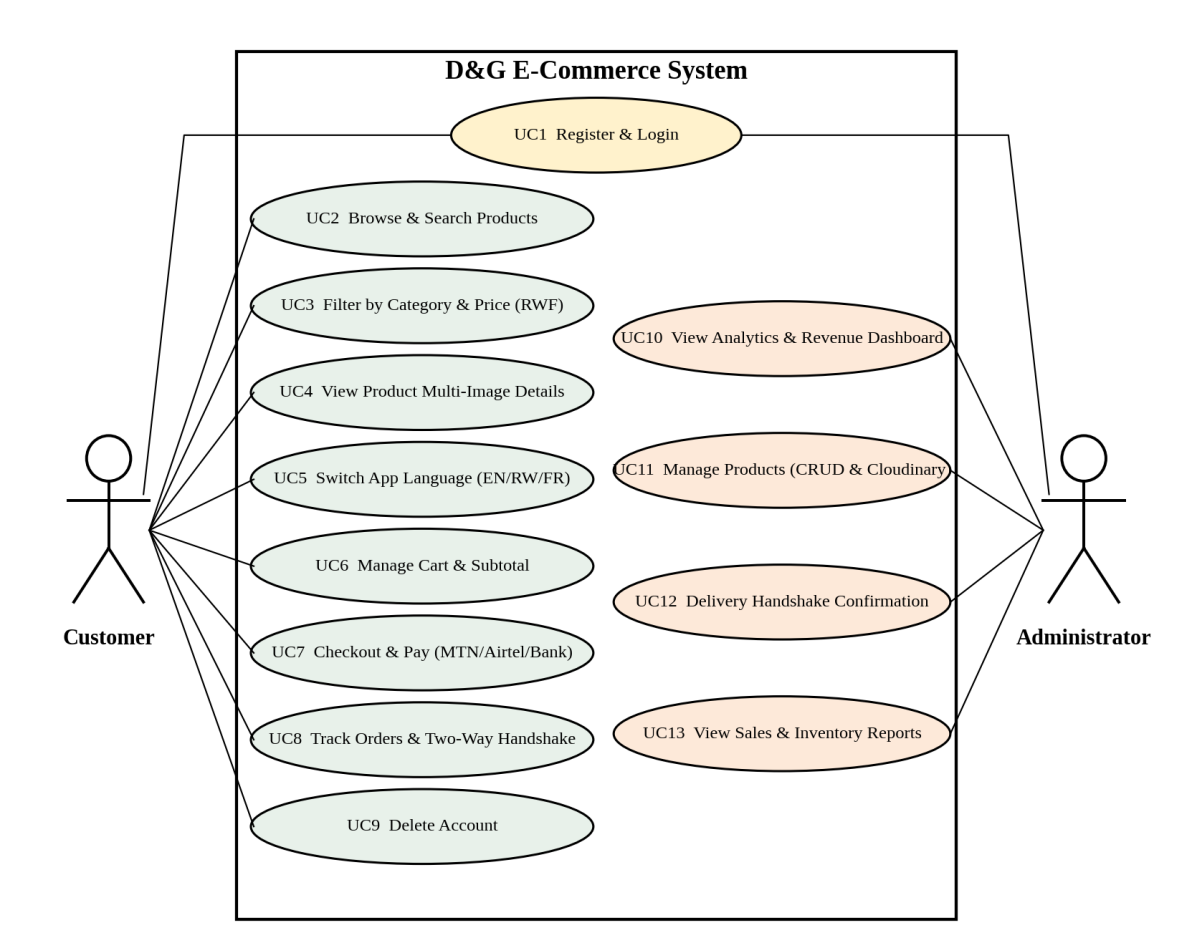


Figure 2: Use case diagram of the D&G application

3.3 Activity Diagrams

The following activity diagrams describe the three most important processes: login, checkout and order tracking.

3.3.1 Login Process

The user enters an email and password. Empty fields trigger a Missing Fields alert. Otherwise the app calls POST /api/auth/login. A failed attempt returns a 401 error and the user stays on the screen. On success the JWT is saved to AsyncStorage, the profile is fetched from GET /api/users/me, and the is_admin flag decides whether the administrator or the customer tabs are loaded.

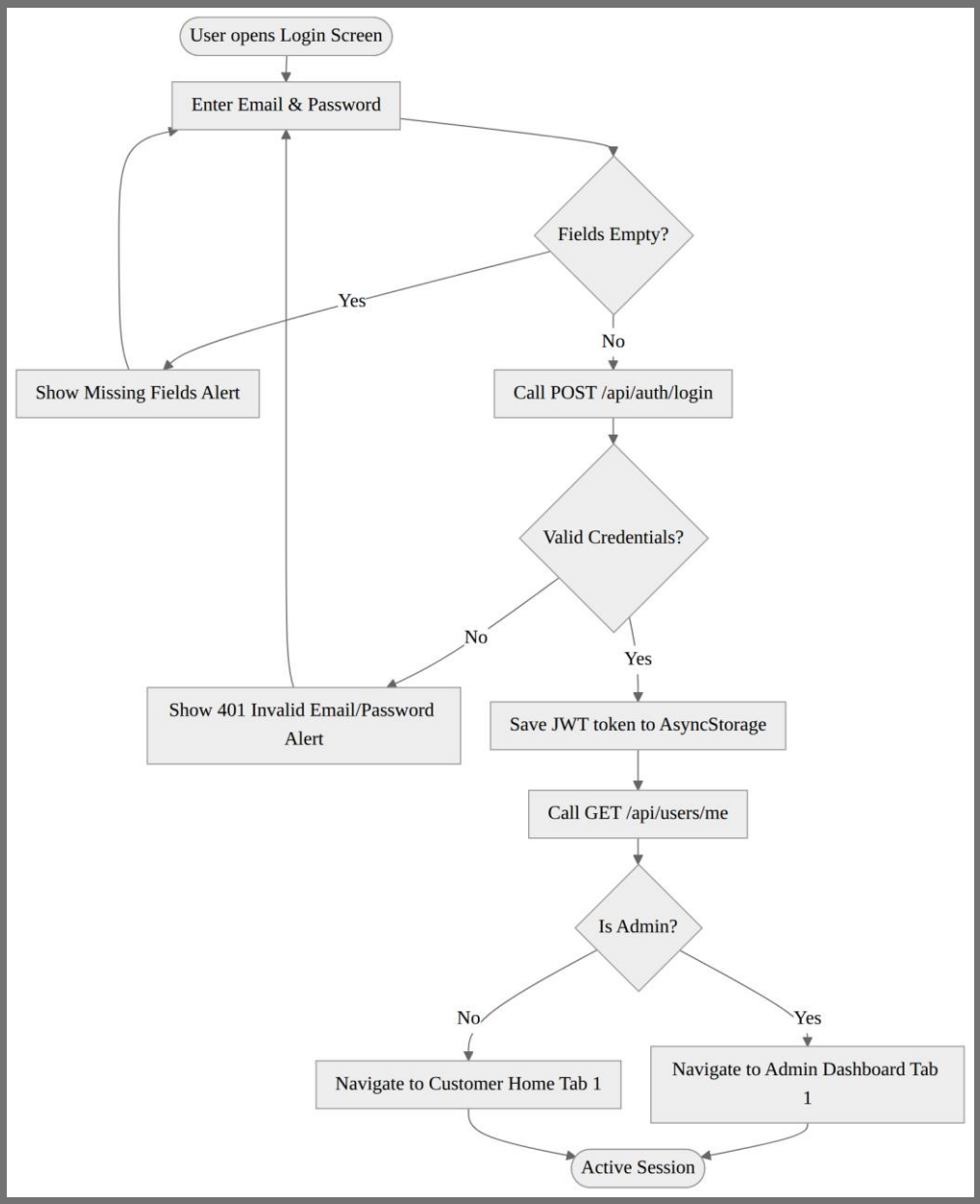


Figure 3: Activity diagram: login process

3.3.2 Checkout Process

Checkout is blocked when the cart is empty. Step 1 validates the name, phone number and district. Step 2 lets the customer choose MTN MoMo, Airtel Money or bank transfer. After the simulated payment is approved, the client posts the order. The backend creates the order, decrements stock, deletes depleted products and clears the cart, after which the client clears its own cart state and shows the confirmation.

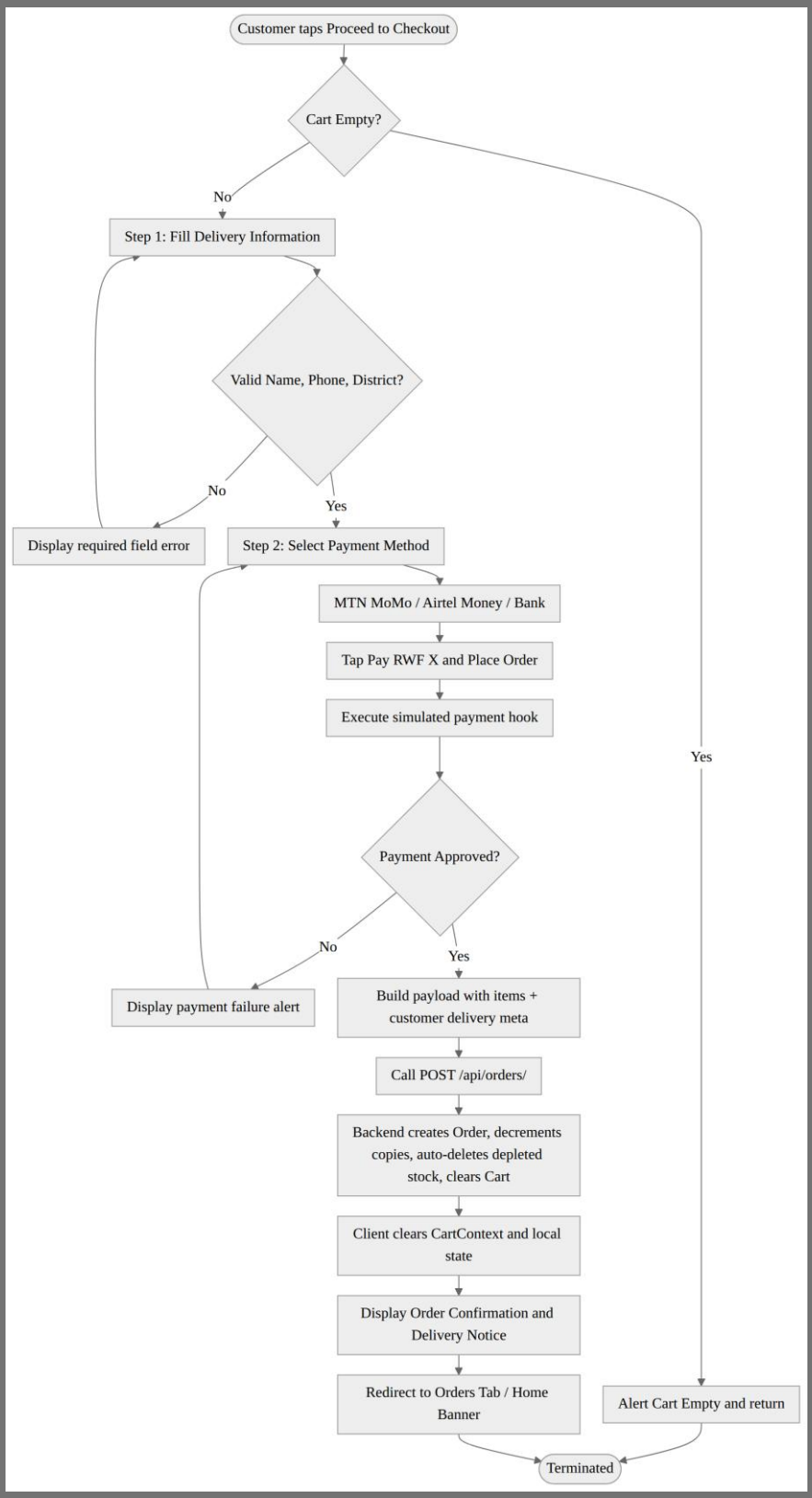


Figure 4: Activity diagram: checkout process

3.3.3 Order Tracking Process

When an order is placed its delivered flag is false and a five-second polling heartbeat starts. The customer sees a Delivery In Transit banner and can confirm receipt; the interface updates

optimistically at once and then calls PATCH /api/orders/:id/deliver. The administrator can confirm delivery from the delivery tab using the same endpoint. The next poll updates every screen to show Delivered.

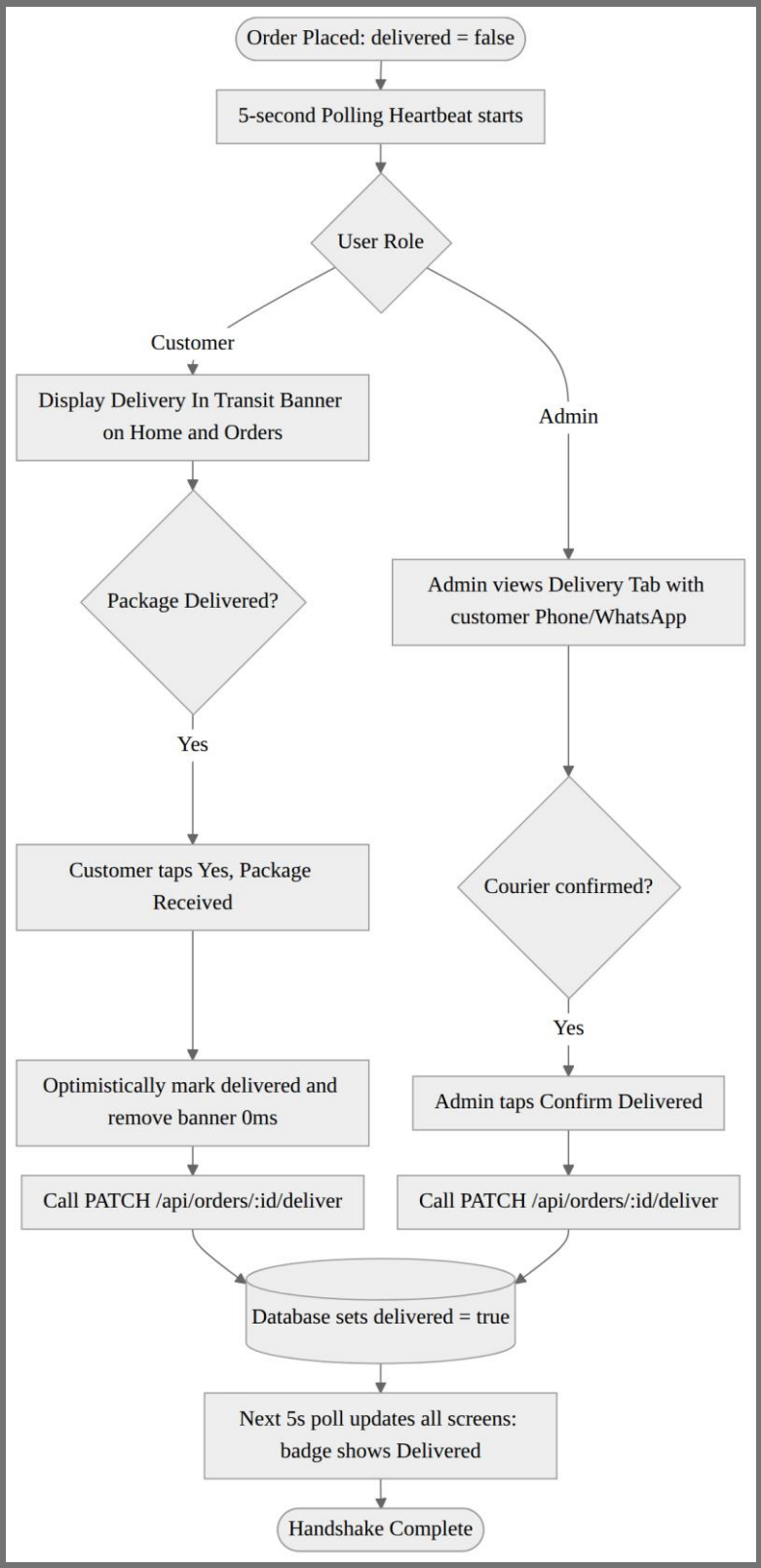


Figure 5: Activity diagram: order tracking process

3.4 Navigation Design

Navigation uses Expo Router. The root stack contains a splash screen with a three-second brand delay, an isolated (auth) stack with Login, Register and Forgot Password, a role-aware (tabs) navigator, and two stack screens that sit above the tabs: the product detail screen and the checkout screen. Each tab file renders either a customer screen or an administrator screen depending on the is_admin flag, so the two roles share one tab bar but see different content.

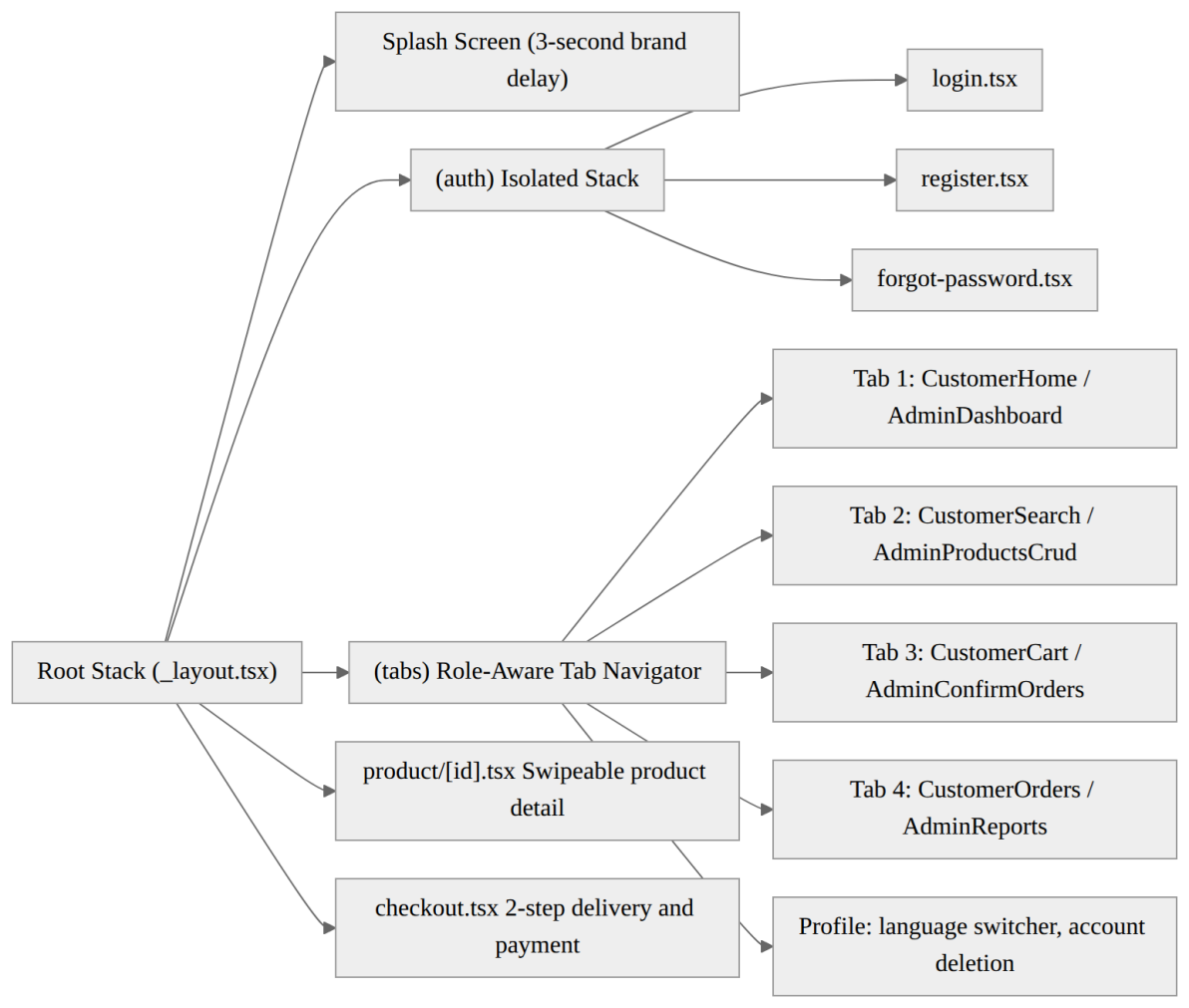


Figure 6: Navigation structure of the application

Table 6 lists every screen with its route, navigator and access rule.

Table 6: Screens and access rules

Screen	File route	Navigator	Login required	Role
Splash	_layout.tsx	Root stack	No	Public
Login	(auth)/login.tsx	Auth stack	No	Unauthenticated
Register	(auth)/register.tsx	Auth stack	No	Unauthenticated
Forgot Password	(auth)/forgot-password.tsx	Auth stack	No	Unauthenticated
Customer Home	(tabs)/tab1.tsx	Bottom tabs	Yes	Customer

Screen	File route	Navigator	Login required	Role
Admin Dashboard	(tabs)/tab1.tsx	Bottom tabs	Yes	Administrator
Customer Search	(tabs)/tab2.tsx	Bottom tabs	Yes	Customer
Admin Products CRUD	(tabs)/tab2.tsx	Bottom tabs	Yes	Administrator
Customer Cart	(tabs)/tab3.tsx	Bottom tabs	Yes	Customer
Admin Deliveries	(tabs)/tab3.tsx	Bottom tabs	Yes	Administrator
Customer Orders	(tabs)/tab4.tsx	Bottom tabs	Yes	Customer
Admin Reports	(tabs)/tab4.tsx	Bottom tabs	Yes	Administrator
Profile and Settings	(tabs)/profile.tsx	Bottom tabs	Yes	All users
Product Details	product/[id].tsx	Root stack	Yes	All users
Checkout	checkout.tsx	Root stack	Yes	Customer

3.5 Data Models

The main objects exchanged with the API are shown below as sample JSON taken from the data models in the backend (models/user.py, models/product.py, models/order.py and schema.sql).

User

```
{
  "id": "c8a41f6e-3472-4d2b-98f9-9dfb5e7d1234",
  "email": "customer@dgretail.rw",
  "is_admin": false
}
```

Product

```
{
  "id": "e4b9812a-89a1-4321-9988-776655443322",
  "name": "Nike Air Max 270",
  "category": "Shoes",
  "price": 125000.0,
  "quantity": 8,
  "info": {
    "description": "Premium breathable running shoes for sports and casual wear.",
    "sizes": ["40", "41", "42", "43"],
    "color": "Black/White"
  },
  "images": [
    "https://res.cloudinary.com/b2h01eeh/image/upload/v1714001/dg/nike_front.jpg",
    "https://res.cloudinary.com/b2h01eeh/image/upload/v1714002/dg/nike_side.jpg"
  ]
}
```

Order

An order stores its items as a list. The last element holds the customer delivery information (marked is_customer_info), which is how the administrator sees phone, WhatsApp, district and address.

```
{
  "id": "ORD-K89FA",
  "user_id": "c8a41f6e-3472-4d2b-98f9-9dfb5e7d1234",
```

```
  "total_price": 250000.0,
  "paid": true,
  "delivered": false,
  "created_at": "2026-10-25T10:15:30",
  "items": [
    { "product_id": "e4b9812a-89a1-4321-9988-776655443322",
      "name": "Nike Air Max 270", "price": 125000.0,
      "quantity": 2, "subtotal": 250000.0 },
    { "is_customer_info": true,
      "customer": {
        "name": "Jean Paul Mugisha", "email": "customer@dgretail.rw",
        "phone": "+250788123456", "whatsapp": "+250788123456",
        "district": "Gasabo", "address": "KG 125 St, Kimironko",
        "payment_method": "MTN_MOMO", "order_ref": "ORD-K89FA" } }
  ]
}
```


3.6 Backend API Overview

Table 7 lists the endpoints of the backend that the application uses. The production base URL is https://dgapp.onrender.com/api.

Table 7: API endpoints used

Endpoint	Method	Access	Purpose
/	GET	Public	Backend health and status check
/api/health	GET	Public	Keep-alive ping
/api/auth/register	POST	Public	Register a user with email and hashed password
/api/auth/login	POST	Public	Authenticate and issue a JWT bearer token
/api/auth/forgot-password	POST	Public	Request a password reset without leaking account status
/api/users/me	GET	Authenticated	Retrieve current profile and role
/api/users/me	DELETE	Authenticated	Cascading deletion of the current account
/api/users/{user_id}	DELETE	Admin or self	Remove a user and all associated records
/api/products/	GET	Public	List products with category and search filters
/api/products/{id}	GET	Public	Retrieve one product
/api/products/	POST	Admin	Create a product with images and stock count
/api/products/{id}	PUT	Admin	Update product details, RWF price and quantity
/api/products/{id}	DELETE	Admin	Remove a product from the catalogue
/api/carts/	GET	Authenticated	Fetch the persistent cart and subtotal
/api/carts/	PUT	Authenticated	Synchronise cart items and total
/api/carts/	DELETE	Authenticated	Empty the cart
/api/orders/	POST	Authenticated	Place an order, decrement stock, clear the cart
/api/orders/	GET	Authenticated	Order history of the current customer

Endpoint	Method	Access	Purpose
/api/orders/all	GET	Admin	All orders with delivery details
/api/orders/{id}/deliver	PATCH	Admin or customer	Two-Way Handshake: mark an order delivered

Chapter 4: Implementation

This is the main chapter of the report. For each module it explains what was built and how it works, shows the key code, and presents screenshots of the working application. Longer code is available in the source code archive and is referenced in Appendix A.

4.1 Project Setup

The frontend was created with Expo SDK 58 and React Native 0.88 using TypeScript and file-based routing with Expo Router. The development environment uses Node.js, the Expo CLI, an Android emulator or device and an iOS simulator. The backend is a FastAPI application in Python 3.11 using SQLAlchemy, deployed on Render with a PostgreSQL database; a local SQLite file (dg.db) is available as a fallback during development.

The source code is organised so that each concern lives in its own folder:

```
DG-Ecommerce/
  Frontend/
    src/
      app/          Expo Router routes: (auth), (tabs), product/[id], checkout
      components/   Reusable UI such as CartItemRow
      constants/    Theme colours (primary #2E7D32, accent #FFB300)
      context/      AuthContext, CartContext, LanguageContext
      screens/      Customer and admin screen components
      services/     api.ts (Axios), imageService.ts (Cloudinary), i18n.ts
  Backend/
    main.py, requirements.txt, schema.sql
    models/       user.py, product.py, order.py
    crud/         crud_order.py and other data operations
```

4.2 Authentication Module

Authentication is implemented as a separate module: its screens live in the isolated (auth) stack, its logic in AuthContext, and its network code in services/api.ts. On a successful login the JWT is stored in AsyncStorage under the key user_token. An Axios request interceptor reads this token and adds the header Authorization: Bearer <token> to every request. After login the app reads user.is_admin to decide which interface to show. The code below shows the Axios instance and interceptor.

```
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const api = axios.create({
    baseURL: process.env.EXPO_PUBLIC_API_URL || 'https://dgapp.onrender.com/api',
    timeout: 60000,
    headers: { 'Content-Type': 'application/json' },
});

// Automatic JWT Bearer injection
api.interceptors.request.use(async (config) => {
```

```
    const token = await AsyncStorage.getItem('user_token');
    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
}, (error) => Promise.reject(error));

export default api;
```

4.2.1 Login

The login screen shows the D&G logo, a Welcome Back card, an email field and a password field with a visibility toggle. Empty fields produce a Missing Fields alert, wrong credentials produce an Invalid Email or Password message, and a successful login routes the user to the customer or administrator home tab. The screen also offers a Register New Account button and a Forgot Password link.

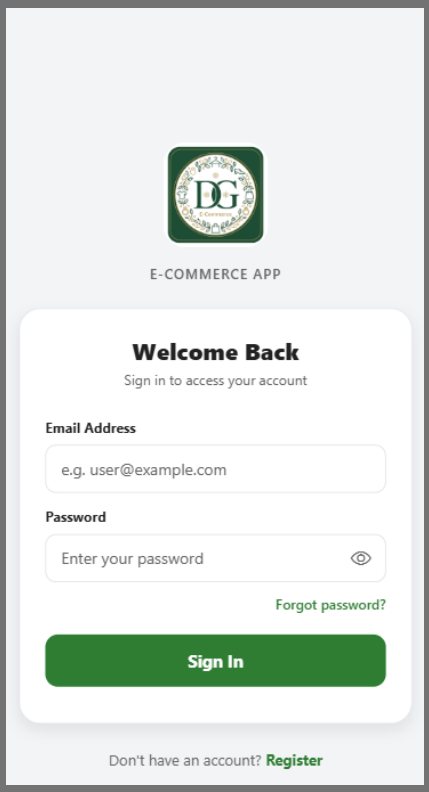


Figure 7: Login screen

4.2.2 Registration

The registration screen collects an email address, a password and a password confirmation. The app checks the email format, the minimum password length and that both passwords match before calling POST /api/auth/register. The API stores only a bcrypt hash of the password.

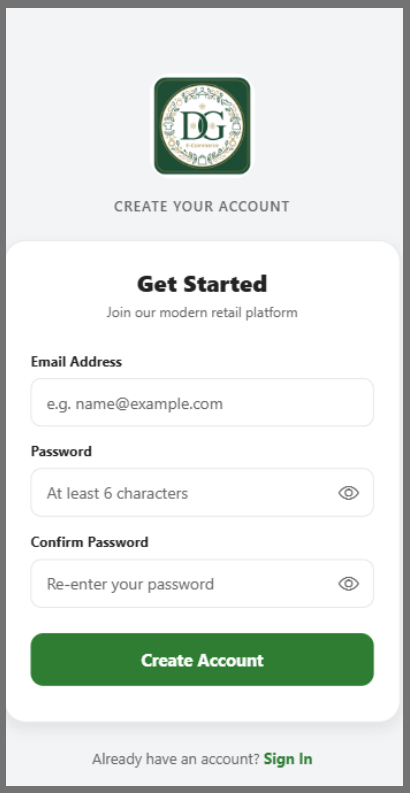


Figure 8: Registration screen

4.2.3 Forgot Password

The user enters the registered email address and taps Send Reset Link, which calls POST /api/auth/forgot-password. The server responds in the same way whether or not the email exists, so the screen cannot be used to discover which addresses are registered.

4.2.4 Logout

Logout removes the token from AsyncStorage, clears the user and cart state held in context and returns the user to the Login screen. Because the tab navigator is only shown for an authenticated user, protected screens are no longer reachable after logout. The profile screen also provides secure account deletion, described in Chapter 6.

4.3 Product Browsing Module

Products are loaded from GET /api/products/ and shown in a two-column grid. The category bar offers All, Clothes, Shoes, Bags, Household, Jewelry and Cosmetics. A second row of chips filters by RWF price tier: Under 50k, 50k-150k and Over 150k. Search is debounced by 300 ms so that the API is not called on every key press.

4.3.1 Product List

Each product card shows a square image with a photo-count badge, the product name, the price formatted in RWF, an amber Only X left badge when stock is low, and an Add to Cart button. When an order is in transit, an amber banner appears at the top of the home screen so the customer can confirm receipt.

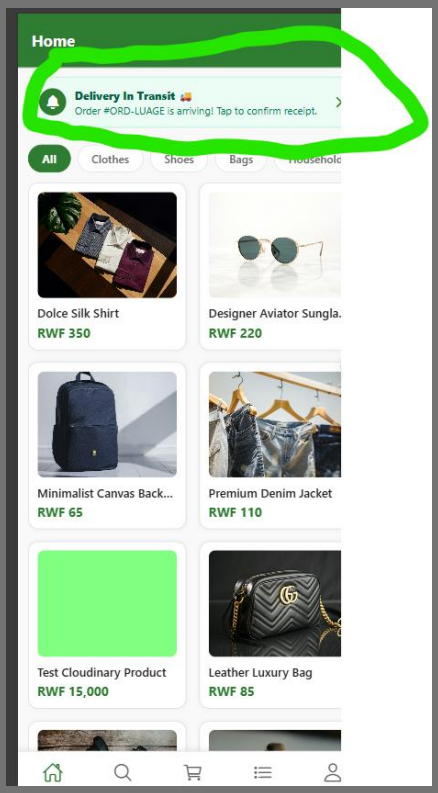


Figure 9: Product list screen

4.3.2 Search

The search box filters products by name as the user types. The effect below waits 300 ms after the last key press before requesting results.

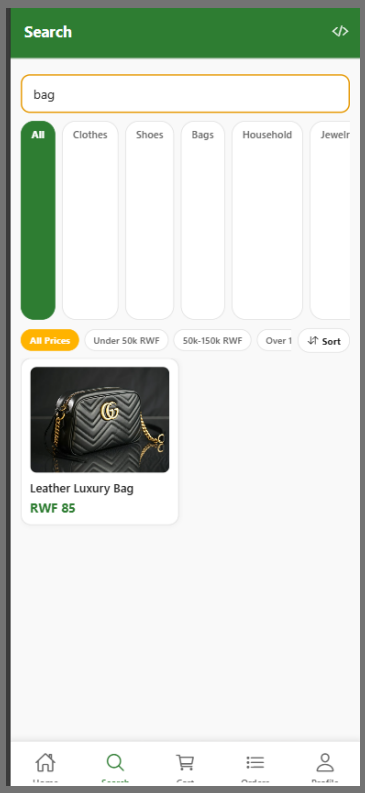


Figure 10: Search by product name

4.3.3 Filter

Category and price filters combine. The following code from CustomerSearch.tsx shows the debounced search and the combined filter logic.

```
useEffect(() => {
    const delayDebounce = setTimeout(() => {
        fetchFilteredProducts();
    }, 300);
    return () => clearTimeout(delayDebounce);
}, [search]);

const filteredProducts = products.filter((p) => {
    const matchesCategory = selectedCategory === 'All' ||
        (p.category || '').toLowerCase() === selectedCategory.toLowerCase();
    const price = parseFloat(p.price || 0);
    let matchesPrice = true;
    if (selectedPriceFilter === 'Under 50k RWF') matchesPrice = price < 50000;
    else if (selectedPriceFilter === '50k-150k RWF')
        matchesPrice = price >= 50000 && price <= 150000;
    else if (selectedPriceFilter === 'Over 150k RWF') matchesPrice = price > 150000;
    return matchesCategory && matchesPrice;
});
```

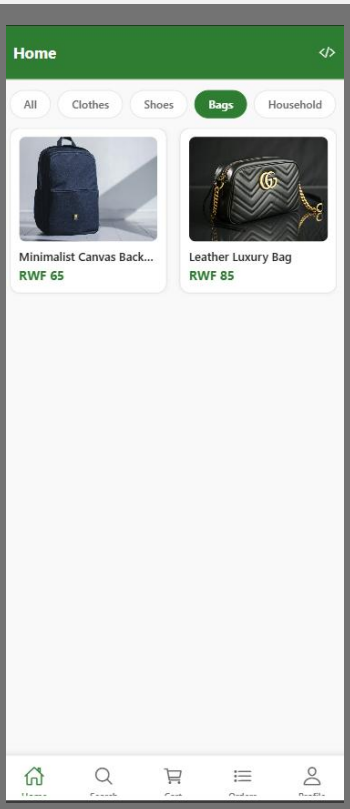


Figure 11: Filtering products by category and price

4.3.4 Product Details

The details screen (product/[id].tsx) shows a full-width swipeable image gallery with pagination dots, the product name, the price in large green text, a category badge, a stock availability pill, the description and a sticky bottom bar with the price and an Add to Cart button.



Figure 12: Product details screen

4.4 Shopping Cart Module

The cart is held in CartContext and synchronised with the backend through PUT /api/carts/, so it survives app restarts and device changes. Every change recalculates the total in RWF. The user can increase or decrease quantities with a stepper, remove an item with the trash icon, and see the subtotal for each line and the total payable.

```
const syncCartWithBackend = async (newItems: any[]) => {
    const newTotal = newItems.reduce(
        (acc, item) => acc + (item.price * item.quantity), 0);
    setCartItems(newItems);
    setTotalPrice(newTotal);
    try {
        await api.put('/carts/', { items: newItems, total_price: newTotal });
    } catch (e) {
        console.error('Failed to sync cart with server', e);
    }
};
```

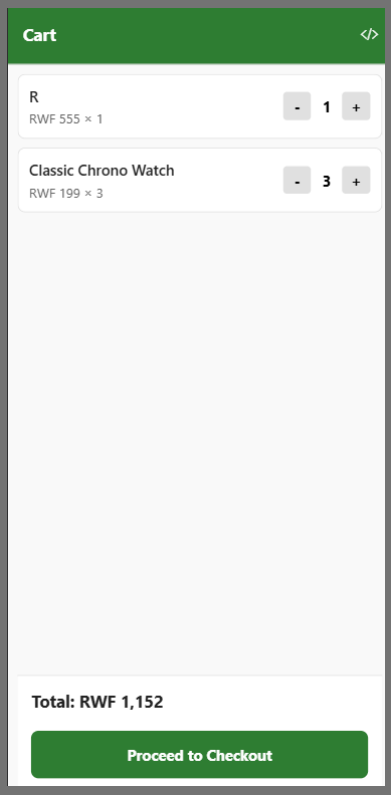


Figure 13: Shopping cart screen

4.5 Checkout Module

Checkout has two steps. Step 1 collects the full name, phone number, WhatsApp number (with a Same as phone number option), delivery district and address notes. The district list contains Rwandan districts such as Gasabo, Kicukiro, Nyarugenge, Rwamagana, Musanze, Rubavu and Huye. Step 2 lets the customer choose MTN Mobile Money, Airtel Money or bank transfer and tap Pay to place the order. Payment is simulated by a hook that approves or rejects the transaction.

When the order is created, the backend decrements the stock of each purchased product and deletes the product if its quantity reaches zero:

```
# Decrement quantities and delete product if copies run out (quantity <= 0)
if order_data.items and isinstance(order_data.items, list):
    for item in order_data.items:
        if not isinstance(item, dict) or item.get('is_customer_info'):
            continue
        prod_id = item.get('product_id') or item.get('id')
        qty = int(item.get('quantity') or 1)
        if prod_id:
            product = db.query(Product).filter(Product.id == str(prod_id)).first()
            if product:
                new_qty = (product.quantity or 0) - qty
                if new_qty <= 0:
                    db.delete(product)
                else:
                    product.quantity = new_qty
db.commit()
```

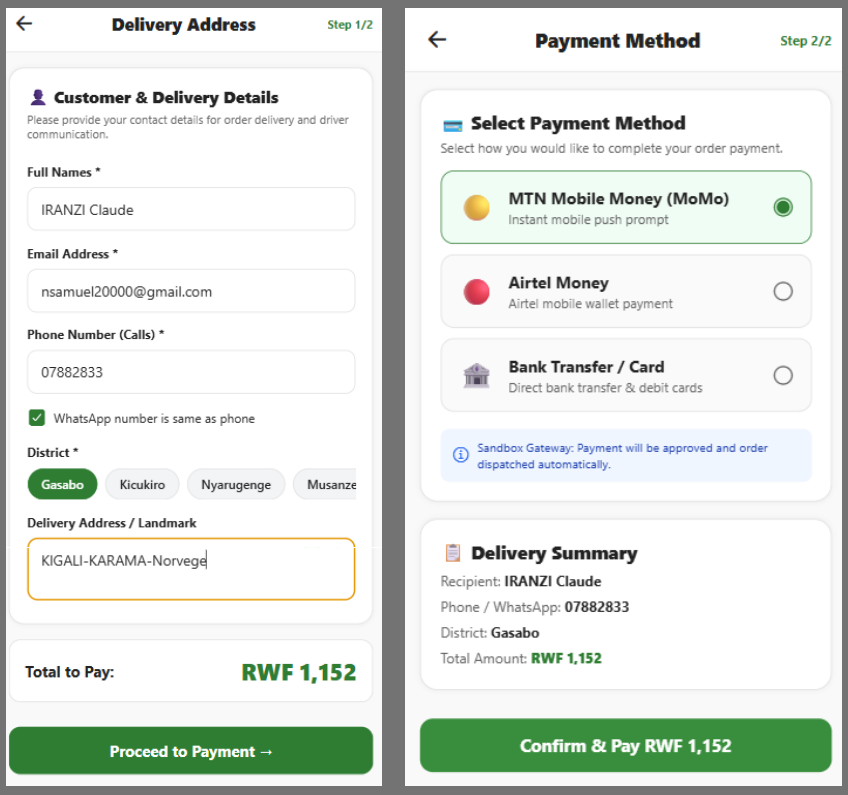


Figure 14: Checkout screen: delivery and payment

4.6 Order Management Module

The Orders tab lists the customer orders with the order reference, date, status badge (amber In Transit or green Delivered), itemised products, total in RWF and, for active orders, a Confirm Package Received button. The screen polls the server every five seconds (POLL_INTERVAL_MS = 5000) so that a delivery confirmed by the administrator appears on the customer device without a manual refresh. When the customer confirms receipt, the interface changes immediately and the request is sent in the background:

```
const handleCustomerConfirmReceipt = async (order: any) => {
    // Optimistically mark as delivered in a blink
    setOrders((prev) => prev.map((o) => (o.id === order.id ? { ...o, delivered: true } : o)));
    try {
        await api.patch(`/orders/${order.id}/deliver`);
        Alert.alert('Success', 'Delivery confirmed! Thank you for shopping with D&G.');
    } catch (e) {
        Alert.alert('Error', 'Could not confirm delivery with server.');
    }
};
```

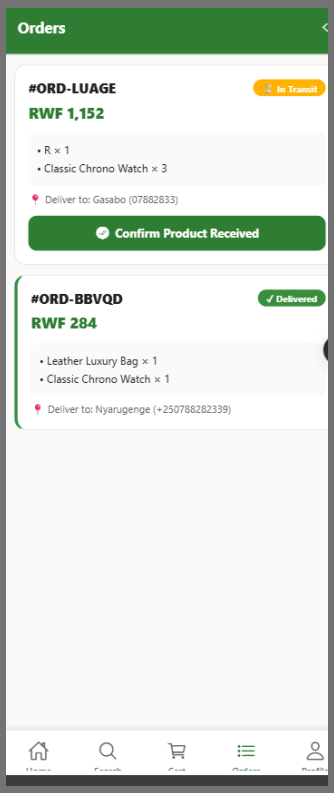


Figure 15: Order history screen

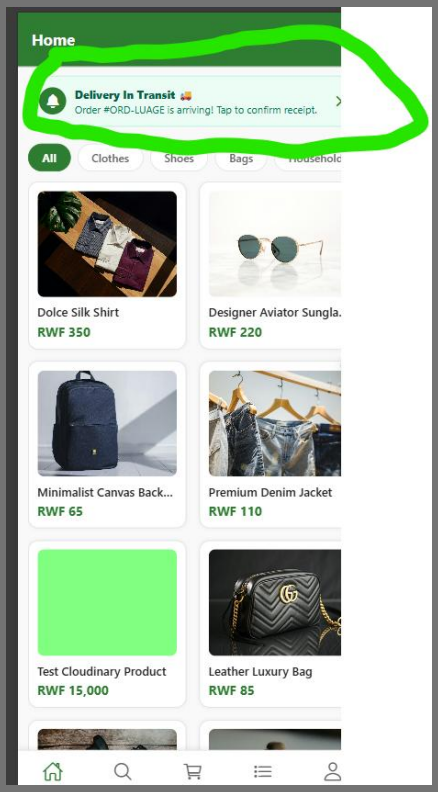


Figure 16: Order details and tracking status

4.7 API Integration

All network communication goes through the single Axios instance shown in Section 4.2, which sets the base URL, a 60-second timeout (long enough to tolerate the cold start of the free Render instance), JSON headers and the bearer token interceptor. The endpoints used are listed in Table 7. Screens show a loading indicator while a request is running and replace stale data silently once the response arrives. Errors are passed through the helper getApiErrorMessage, which converts server and network errors into short, readable messages. Product images are uploaded straight from the device to Cloudinary by imageService.ts using the unsigned preset DGAppImages; the upload sends the picture as Base64 JSON, which avoids multipart boundary problems on Android (see Chapter 6).

```
export async function uploadImageToCloudinary(input: string | ImageUploadInput): Promise<string |
null> {
    const CLOUD_NAME = "b2h01eeh";
    const UPLOAD_PRESET = "DGAppImages";
    // Base64 upload to bypass Android multipart boundary bugs
    const response = await fetch(`https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            file: `${input.base64}`,
            upload_preset: UPLOAD_PRESET,
        }),
    });
    const result = await response.json();
    return result.secure_url || null;
}
```

4.8 Multi-language Support

The interface is available in English (en), Ikinyarwanda (rw) and French (fr). A central translation dictionary in i18n.ts holds every string, and LanguageContext exposes the selected language and a translation function t to all screens. Changing the language re-renders the interface immediately without reloading the app, and the choice is kept after the app restarts. A helper formatRWF() formats every price consistently.

```
export function formatRWF(amount: number): string {
    if (isNaN(amount) || amount === null || amount === undefined) {
        return 'RWF 0';
    }
    return `RWF ${Math.round(amount).toLocaleString('en-US')}`;
}
```

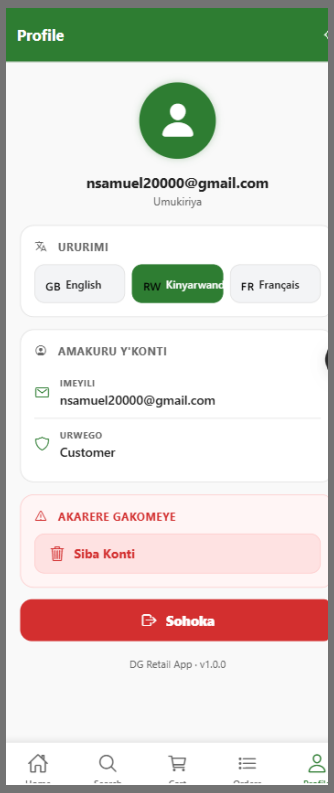


Figure 17: Language switching: translated version (Ikinyarwanda)

4.9 Navigation and Access Control

The root layout (_layout.tsx) shows the splash screen and then decides which stack to display. Unauthenticated users can reach only the (auth) stack. Once a token and profile are available, the (tabs) navigator is shown and the (auth) screens are no longer reachable. Within the tabs, each file renders a customer screen or an administrator screen according to user.is_admin, so an administrator never sees customer checkout screens and a customer never sees management screens. The tab bar height is computed as 54 plus the bottom safe-area inset so that icons are not hidden by Android soft keys or the iOS home indicator. The access rules for each screen are given in Table 6.

4.10 State Management

State is managed with React Context, complemented by module-level caches. AuthContext holds the user and token, CartContext holds the cart items and total, and LanguageContext holds the selected language. Each Customer and Admin tab screen also keeps its last fetched data in module memory, so returning to a tab shows data instantly (0 ms) while a silent background request refreshes it. Table 8 summarises the state layers.

Table 8: State management contexts

State context	State held	Main actions	Persistence
AuthContext	user, token, isLoading	login(), register(), logout(), deleteAccount()	AsyncStorage (user_token) and backend /users/me

State context	State held	Main actions	Persistence
CartContext	cartItems, totalPrice	addToCart(), updateQuantity(), clearCart(), fetchCart()	Backend /api/carts/ and in-memory context
LanguageContext	lang (en, rw or fr), t	setLang(newLang)	React context (in memory)
Tab in-memory caches	cachedProducts, cachedOrders, cachedStats	Silent background synchronisation	Module memory (zero-spinner tab switching)

4.11 Validation, Error Handling and User Feedback

The application validates input before it contacts the server and gives clear feedback at every step. Examples are: Missing Fields (please enter both email and password), Required Field (please enter your contact phone number) and Password Mismatch (passwords do not match). Failed requests are converted by getApiErrorMessage into friendly alerts with a retry option, loading indicators are shown while requests are running, and an offline banner with a retry button appears when the connection is lost. Before an order is sent, checkout.tsx also checks that the requested quantities are still in stock.

Chapter 5: Reports Generated by the System

This chapter presents the reports that the application produces from the data supported by the API. Customers see cart and order reports, and administrators see catalogue, customer, order and sales reports.

5.1 Product Listing Report

Generated from GET /api/products/ and displayed in AdminProductsCrud.tsx. It lists the product name, category, unit price in RWF, available stock quantity, an image-count badge and a Low Stock pill (Only X left!) for each product, as cards with a thumbnail, an edit button and a delete button.

5.2 Customer Information Report

Built from the order metadata in AdminConfirmOrders.tsx. For each order it shows the customer full name, account email, phone number, WhatsApp number, delivery district, delivery address, selected payment channel and order reference. The administrator uses it to contact the customer and arrange delivery.

5.3 Cart Summary Report

Generated in CustomerCart.tsx before checkout. It lists each selected product with its thumbnail, unit price in RWF, quantity stepper and item subtotal, and a highlighted bottom bar with the total payable amount.

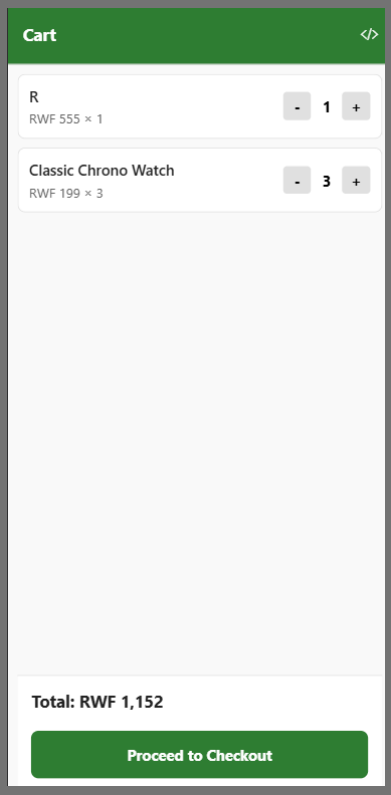


Figure 18: Cart summary report

5.4 Order History Report

Rendered in CustomerOrders.tsx (and AdminConfirmOrders.tsx for administrators). It shows the order reference such as #ORD-7A9B, the order date, a status badge (In Transit in amber, Delivered in green), the itemised products with unit prices, the total in RWF and the receipt confirmation button.

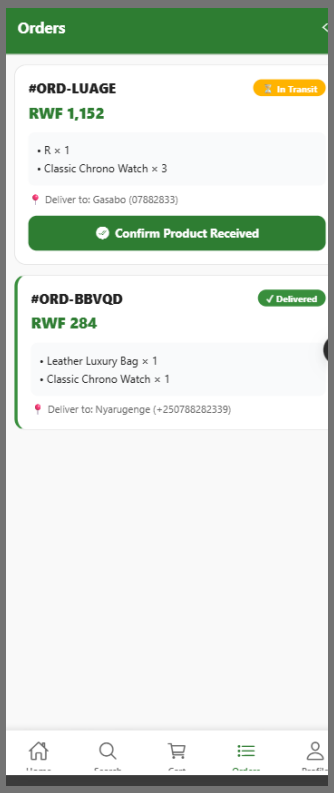


Figure 19: Order history report

5.5 Sales Summary Report

The executive analytics dashboard is produced by AdminReports.tsx and AdminDashboard.tsx. It calculates the following metrics from the order and product data:

- Total delivered revenue: the sum of all orders where delivered is true, formatted in RWF.
- Fulfilment rate: delivered orders divided by total orders, multiplied by 100.
- Average order value: total delivered revenue divided by the number of delivered orders.
- Catalogue distribution: the number of products and the total stock value for each category (Clothes, Shoes, Bags, Household, Jewelry and Cosmetics).

5.6 Application Performance Report

Measured performance of the production build is summarised below. Detailed timings per screen are in Chapter 4 of the Testing Report.

- Tab load latency: 0 ms for cached tab views (instant navigation without spinners).
- API round-trip latency: between 180 ms and 320 ms on the Render cloud backend.
- Image delivery: served from the Cloudinary CDN edge cache with responsive WebP compression, in under 150 ms.
- Memory footprint: stable at about 42 MB of RAM on Android devices.

Chapter 6: Challenges and Solutions

Several technical problems appeared during development. Table 9 describes the most important ones, their causes and how they were solved.

Table 9: Challenges and solutions

Challenge	Cause	Solution
Android soft navigation inset clipping	On Android devices with on-screen back and home buttons, a fixed tab bar height left the icons partly covered.	Used useSafeAreaInsets() in _layout.tsx and calculated the tab bar height dynamically (tabHeight = 54 + insets.bottom).
Android Cloudinary upload failures	Standard FormData uploads on React Native Android can fail when a custom Content-Type header removes the automatic multipart boundary.	Added Base64 JSON upload to imageService.ts, sending the image directly to the Cloudinary API.
Inventory overselling and sold-out items staying visible	After the last unit was bought, the product stayed in the catalogue and could be ordered again.	Added automatic stock decrement and deletion at zero in crud_order.py, with client-side stock pre-validation in checkout.tsx.
Delivery status desynchronisation	Customers kept seeing In Transit after delivery, and administrators had no proof of delivery.	Built the Two-Way Delivery Handshake with five-second polling and optimistic updates so both interfaces stay synchronised.
Loading spinners when switching tabs	Every tab change reloaded data and showed a blank spinner.	Added in-memory module caches (cachedProducts, cachedOrders) that display previous data instantly while refreshing silently.
Account deletion without re-authentication	Anyone with access to an unlocked phone could delete the account.	Added a security modal in profile.tsx that asks for email and password and verifies them through /auth/login before sending the cascade delete.

Chapter 7: Conclusion and Recommendations

7.1 Conclusion

The project set out to give D&G a secure, localised mobile shopping channel. All seven specific objectives were achieved. The application lets customers register, browse and filter products in three languages, pay in RWF through simulated Rwandan channels, and track deliveries through a synchronised handshake, while administrators manage the catalogue, deliveries and sales reports. The code compiles with zero TypeScript errors and the Testing Report records 44 of 44 test cases passed on the final build. Table 10 compares each objective with the evidence in the system.

Table 10: Objectives versus achievements

Specific objective	Status	Evidence in the implemented system
User authentication	Achieved	Isolated auth stack with login, register, forgot password, JWT storage in AsyncStorage and cascading account deletion (AuthContext.tsx, login.tsx).
Product browsing and search	Achieved	Category filtering, debounced search, RWF price tiers, multi-image Cloudinary viewing and three-language switcher (CustomerHome.tsx, CustomerSearch.tsx, [id].tsx).
Shopping cart and checkout	Achieved	Persistent cart with quantity stepper, two-step checkout with district selector and simulated MTN MoMo and Airtel payment (CartContext.tsx, checkout.tsx).
Order tracking and handshake	Achieved	Live order history and Two-Way Delivery Handshake with five-second polling and optimistic confirmation (CustomerOrders.tsx, AdminConfirmOrders.tsx).
Inventory control	Achieved	Stock tracking, automatic decrement on purchase and deletion at zero stock (crud_order.py).
API integration	Achieved	Central Axios instance with JWT interceptor, error normalisation and Render deployment (api.ts).
Administrative operations	Achieved	Dashboard with revenue KPIs, catalogue CRUD with Cloudinary multi-image picker and delivery management (AdminDashboard.tsx, AdminProductsCrud.tsx).

Limitations: payments are simulated and no live mobile money gateway is connected; the free Render backend can take about ten seconds to wake after inactivity, which is reduced by a warm-up request at launch; and the JWT is stored in AsyncStorage, which is not encrypted storage.

7.2 Future Improvements

- Live merchant integration with the MTN MoMo Open API and the Airtel Money Open Gateway.
- Push notifications with Expo Notifications for instant delivery updates.
- Customer reviews, ratings and a wishlist.
- GPS live courier tracking on a map.
- Moving token storage to expo-secure-store for encrypted storage on the device.

References

Axios (n.d.) Axios documentation. Available at: https://axios-http.com/docs/intro (Accessed: 25 October 2026).

Cloudinary (n.d.) Upload API reference. Available at: https://cloudinary.com/documentation/image_upload_api_reference (Accessed: 25 October 2026).

Expo (n.d.) Expo and Expo Router documentation. Available at: https://docs.expo.dev (Accessed: 25 October 2026).

Jones, M. and Hardt, D. (2012) The OAuth 2.0 Authorization Framework: Bearer Token Usage, RFC 6750. Internet Engineering Task Force.

Jones, M., Bradley, J. and Sakimura, N. (2015) JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force.

Meta Platforms (n.d.) React Native documentation. Available at: https://reactnative.dev/docs/getting-started (Accessed: 25 October 2026).

PostgreSQL Global Development Group (n.d.) PostgreSQL documentation. Available at: https://www.postgresql.org/docs/ (Accessed: 25 October 2026).

Ramirez, S. (n.d.) FastAPI documentation. Available at: https://fastapi.tiangolo.com (Accessed: 25 October 2026).

SQLAlchemy (n.d.) SQLAlchemy 2.0 documentation. Available at: https://docs.sqlalchemy.org (Accessed: 25 October 2026).

Appendices

Appendix A: Important Code Snippets

The short snippets in Chapter 4 are taken from the files below. The complete files are included in the submitted source code archive.

Table 11: Key source files

No.	Description	File
1	Root layout with splash screen and inset handling	Frontend/src/app/_layout.tsx
2	Authentication context with JWT storage	Frontend/src/context/AuthContext.tsx
3	Shopping cart provider with server synchronisation	Frontend/src/context/CartContext.tsx
4	Order, stock and delivery handshake backend logic	Backend/crud/crud_order.py
5	Cloudinary unsigned upload utility	Frontend/src/services/imageService.ts

Appendix B: Installation Guide

Unzip the submitted source archive and follow these steps. The cloud API is live, so the backend steps are optional.

```
# 1. Open the project folder
cd DG-Ecommerce

# 2. Frontend setup and execution
cd Frontend
npm install
npx expo start
```

```
# Press a for Android emulator or device, i for iOS simulator, w for web

# 3. Backend setup (optional)
cd ../Backend
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

……...END ……..